Compound Operators: ++, --, +=, -=, *=, /=

In the previous examples, we needed two values for the operators to work their magic, as in the following statement:

```
myValue = 0;
myValue = myValue +1;
```

Instead, we could use compound operators that combine an arithmetic operation with an assignment operation, thereby only needing one variable statement to do what they do. These can be used to either **increment**, raise the value, or **decrement**, lower the value, using just one operator. The following is an example that increments `myValue` by 1 using the `++` character symbol:

```
myValue = 0;
myValue++;
```

This statement, functionally identical to the very last code example, increments the variable by 1 and reassigns the new value to the current variable name. In this case, the variable `myValue` now contains the numerical value 1. Likewise, we can decrement the value using the `--` character symbol, as follows:

```
myValue--;
```

Here, `myValue` now contains 0 again. In these two examples, the compound operators `++` or `--` only increment or decrement by a value of 1. If we wanted to increment or decrement a variable by a value greater than 1 we could use one of these compound operators: `+=`, `-=`, `*=`, or `/=`. For example if we wanted to increment `myValue` by 10 we could use the following statement:

```
myValue += 10;
```

This statement, again identical to the statement `myValue = myValue +10;` adds 10 to whatever value is assigned to `myValue`. Assuming that `myValue` contains the number 10, we can multiply it by a factor and reassign the new value using the following example:

```
myValue *= 1.5;
```

The variable `myValue` has now been increased to 15. Remember though, if we assume `myValue` is an integer, a further `*= 1.5` would result in 22 rather than 22.5 because integers lob off anything after the decimal point.

We used compound operators in our recent project code to multiply two variables together to create a longer delay time for white, as in the following example:

```
delay(time*=multiple);
```

In this line of code, we multiply the value of the variable `multiple` by the current value of `time`, assign this new value back to the variable `time`, and then we delay for that amount. In this way we can change either of the initial values for these two variables at the beginning of the code without needing to hunt down each line in the sketch. Compound operators are also useful for counters and indices and especially help compact the code into a more readable form when we get to the `for` loop control structures later.

Order of Operations

When the math gets a little more complicated, it's good to know about the **order of operations**, or the order in which operators are processed in any given expression. For example, depending on the way an expression is evaluated, `3 * 5 + 10` could yield either 25 or 45 depending on whether we multiply 3 by 5